



۸۱۰۱۰۱۴۹۲



نوت ۵ آذر

شبکه‌های کامپیوتری
نام و نام خانوادگی: علیرضا کریمی

فصل چهارم: لایه *Network* بخش *Data – Plane* اسلاید ۱۱ تا ۶۱ [۱]

معماری روتر

روتر دارای پورت ورودی و چند پورت خروجی است. اتصال پورت‌ها به هم با استفاده از ساختار سویچ سریع *highspeed switching fabric* انجام می‌شود. یک پردازنده در روتر این الگوریتم روتینگ را اجرا می‌کند. الگوریتم‌های روتینگ هر چند میلی ثانیه یک بار اجرا می‌شوند اما عمل فورواردینگ هر چند نانو ثانیه یک بار انجام می‌شود. در ساختار پورت ورودی و خروجی به نحوی است که پروتکل‌های لایه فیزیکی و دیتالینک و شبکه را می‌تواند اجرا کند.

ساختار پورت ورودی

در بخش مربوط به لایه شبکه، هر پورت عمل *lookup* و *forwarding* را انجام می‌دهد. صف هم داریم. پردازش روتینگ معمولاً جدول فورواردینگ را در هر پورت قرار می‌دهد و طبق آن *match* یا تطبیق می‌کنیم که آدرس مقصد هر بسته مربوط به کدام پورت خروجی است. بسته به آن پورت منتقل می‌شود که به این *action* می‌گویند. هدف این است که پردازش فورواردینگ در سطح *line speed* باشد؛ یعنی نرخ ورود بیشتر از نرخ پردازش نباشد تا بسته‌ها در صف قرار نگیرند.

- در روش اول روتینگ، این روتینگ طبق آدرس مقصد انجام می‌شود.
 - در روش مبتنی بر نرم‌افزار که به آن *generalized forwarding* نیز می‌گویند، از فیلدهای دیگر هدر برای فورواردینگ استفاده می‌شود.
- روترها یک دامنه آدرس متوالی (فرضا مربوط به یک ناحیه جغرافیایی) را با استفاده از *address aggregation* به صورت یک محدوده آدرس واحد در نظر گرفته و برای آن، یک پورت خروجی یکسان اختصاص می‌دهند. قسمت نامشترک از آدرس شروع و آدرس پایان مربوط به یک مدخل را ستاره می‌گذاریم. اگر آدرس مقصد اینگونه با دو مدخل متناظر باشد، به مدخلی با *longest prefix match* منتقل می‌شود.
- برای بازیابی آدرس با سرعت بالا در جداول فورواردینگ، آنها را با تکنولوژی حافظه *TCAM* پیاده‌سازی می‌کنند.

این حافظه در یک سیکل آدرس را *retrieve* می‌کند.

ساختار سوییچ در روتر *switching fabrics*

وظیفه این ساختار، هدایت بسته از بافر پورت ورودی به بافر پورت خروجی است.

سرعت آن با نرخ سوییچ کردن یا فورواردینگ مشخص می‌شود که ضربی از نرخ خط است. دوست داریم برای ورودی n تایی، نرخ سوییچ کردن، n برابر سرعت هر خط باشد.

تا در هر *round* بسته‌های همه خط‌های ورودی را به خروجی مناسب هدایت کنیم.

سه روش پیاده‌سازی ساختار سوییچ:

• روش اول: به کمک یک کامپیوتر معمولی. بسته ورودی توسط *cpu* به حافظه منتقل شده و از آن جا به پورت خروجی منتقل می‌شود. پس ۲ تا *IO* است و پهنای باند حافظه محدود و وابسته است.

• روش دوم: استفاده از گذرگاه بورد اصلی کامپیوتر *bus*. پورت‌های ورودی و خروجی حافظه مستقل دارند. بسته‌ها از طریق باس از حافظه پورت ورودی به پورت خروجی می‌روند و در هر لحظه فقط بسته یک پورت ورودی به پورت خروجی می‌رود. اگر بسته‌های پورت‌های دیگر بخواهند منتقل شوند، به مشکل *bus contention* می‌خوریم. پس پهنای باند باس، سوییچینگ را محدود می‌کند.

• روش سوم: استفاده از شبکه‌های میان‌ارتباطی در روترها. پورت‌های ورودی در سطر و پورت‌های خروجی در ستون هستند و می‌توان حداکثر تا n ورودی را به n خروجی به‌طور همزمان وصل کرد؛ مشروط بر اینکه هر ورودی با یک خروجی مجزا ارتباط بگیرد.

بهینه‌سازی‌ها: شکستن بسته‌ها به سلول‌هایی و منتقل کردن آنها: موسوم به *cell switching*

باید سرعت عبور بسته‌ها از ساختار سوییچ بیشتر از مجموع ورودی‌ها از کل پورت‌های ورودی باشد وگرنه بسته‌ها باید در صف قرار بگیرند. ممکن است یک بسته سر صف باشد و بخواهد مثلاً به پورت خروجی ۱ برود و نمیتواند اما بسته پشت آن میخواهد به پورت خروجی دیگری برود و نمیتواند. این مشکل *HOL blocking* است. یعنی اونی که سر صف است، مانع رفتن بقیه اون صف به پورت‌های خروجی (حتی پورت‌های دیگر) میشود.

همینطور که در پورت ورودی بافر داشتیم، در پورت خروجی هم داریم. زمانی که سرعت رسیدن دیتاگرام از ساختار سوییچ، سریع تر *transmission* آنها باشد، دیتاگرام‌ها در بافر پورت خروجی جمع می‌شوند. حال وقتی بسته‌ها در بافر پورت خروجی جمع شدند، باید بدانیم که میخواهیم طبق چه *priority* آن‌ها را ترخیص کنیم. یعنی طبق چه *scheduling discipline*.

مثلاً اگر نظام زمان بندی ما با تقدم باشد، ممکن است بسته ای را زودتر مرخص کنیم که تاثیر بیشتری روی *performance* دارد.

البته این وجود تقدم در شبکه میتواند بی طرفی شبکه را تحت الشعاع قرار دهد. و متناقض با آن است.

اگر هم بافر پورت خروجی پر شود، *loss* خواهیم داشت از بعد آن. اینکه بافر در کجا باشد و چقدر باشد، مورد بحث است. مثلاً فقط در پورت هی ورودی یا خروجی یا هر دو باشد. بعضی‌ها هم در مورد اندازه آن صحبت کردند. مثلاً در RFC 3439 گفتند که این اندازه برابر با $RTT \times C$ که $C = \text{link capacity}$ است. جدیداً می‌گن *flow* و تعداد ورودی‌ها را هم در نظر بگیریم. و اینگونه مقدار بافر برابر می‌شود با: $\frac{RTT \times C}{\sqrt{N}}$

همانطور که گفتیم، برای خروج بسته‌ها اولویت داریم مثلاً، *FIFO* حال اینکه بافر پر شد و بسته جدیدی آمد، کدام بسته را حذف کنیم، باز پیاده‌سازی‌های مختلفی دارد. مثلاً همانی که الان آمده را حذف کنیم، یا اینکه طبق یک اولویت خاصی این کار را انجام دهد، یا رندوم یکی را حذف کند (با در نظر گرفتن بسته جدید)

برای زمان بندی با تقدم، به نوعی میتوان به بسته‌ها مارک زد و مثلاً با کمک فیلدهای مختلف در هدر، آنها را اولویت بندی کرد. مثلاً از روی پورت و آدرس مقصد یا مبدا و غیره. فکر کنم اگر این اولویت بندی به صورت *non-preemptive* باشد، بسته با اولویت کمتر همیشه ممکن است در صف بماند.

اگر زمان بندی *round-robin* باشد، از هر تقدم به ترتیب به صورت دوری، یکی را عبور میدهد و اگر از اون تقدم نباشد، از آن عبور میکند.

اگر به صورت *Weighted fair queuing* باشد که همان تعمیم یافته *round-robin* است، و به هر کلاس وزن تخصیص میدهد و در هر دور طبق وزن کلاس، بسته از آن کلاس به خروجی میرود.

IP

خب به نوعی پشته استک شبکه، با حالت ساعت شنی و کمر باریک است چون همه در لایه شبکه از پروتکل *IP* استفاده میکنند. این پروتکل مربوط به قالب پیام و نحوه برخورد با داده‌ها و بدست آوردن آدرس آنهاست. پروتکل‌های روتینگ هم برای مسیریابی به کار می‌روند. پروتکل *ICMP* هم برای گزارش ارور و یا فرستادن سیگنال از روترها به کار میرود.

هدر مربوط به *IP* دیتاگرام، حداقل ۵ تا ۳۲ بیت یا در مجموع ۲۰ بایت است. یک فیلد $TTL = \text{time to live}$ است که به هر روتر که برسد یکی از آن کم می‌شود. فیلد *upper-layer* مشخص میکند که باید به چه پروتکلی در لایه بالاتر، *payload* را تحویل دهد. ۳ تا فیلد ردیف دوم هم برای *fragmentation* یا همان تکه‌سازی بسته‌ها و *reassemble* آنها کاربرد دارد.

یک فیلد *option* هم داریم که میتواند طول هدر را متغیر کند تا حدی. باید سعی کنیم که طول هدر نسبت به طول مفید داده، زیاد نباشد.

IP fragmentation and reassembly

خب هر لینک طبق لایه دیتا لینک، یک حداکثر ظرفیت انتقال حجم دارد که به $MTU = \text{maximum transfer unit}$ معروف است. یعنی اگر بسته های IP از آن بزرگ تر باشند، باید طبق آن تکه تکه شوند و به هر کدام هدر اضافه شود و به مقصد بروند. در مقصد از روی هدر آنها، بسته اصلی ساخته می‌شود.

مثلا وقتی یک بسته داریم به اندازه ۵۰۰۰ بایت با ۴۹۸۰ بایت داده و ۲۰ بایت هدر، و مثلا $MTU = 1500$ است. پس باید به سه تا بسته ۱۴۸۰، ۱۴۸۰ و ۱۰۲۰ تقسیم شود که هر کدام هم ۲۰ بایت برای هدر دارند. فیلد ID این بسته های تکه شده برابر x است. فیلد $fragflag$ همه این بسته ها به جز آخری، ۱ است. آخری صفر است. در نهایت فیلد سوم، فیلد $OFFSET$ است که یعنی از کجا شروع میشود و برابر تعداد بایت قبل از آن تقسیم بر ۸ است. یعنی بسته اول $OFFSET$ ش صفر است. بسته دوم ۱۴۸۰/۸ است.

آدرس دهی IPv4:

این آدرس ۳۲ بیت است و به ازای هر $interface$ تعریف میشود. هر $interface$ محل اتصال بین $host$ و روترها با لینک فیزیکی است.

روترها چندتا $interface$ دارن و هاست ها ۱ یا ۲ تا دارند.

این ۳۲ بیت را ۴ تا ۸ بیتی میکنن و با دات جدا میکنند و هر کدام از آن ۸ بیت را به صورت دسیمال نشون میدن. و به این دات دسیمال میکنند. دسیمال هایی که با نقه از هم جدا شدند.

$interface$ ها چگونه در یک جزیره به هم وصل میشوند؟ بستگی به تکنولوژی دارد. اگر از تکنولوژی $Ethernet$ استفاده کند، با سوییچ $Ethernet$ و اگر با تکنولوژی بی سیم مثل وای فای باشد، از طریق $wifi base station$ به هم وصل میشوند.

زیر شبکه میشه همه دستگاه های متصل شده به یک $interface$ روتر که تشکیل یک جزیره میدهند. دارای بخش آدرس زیرشبکه یکسان هستند. این دستگاه ها میتوانند بین خودشان اطلاعات را بدون عبور از روتر، انتقال دهند.

۲۴ بیت اول (سمت چپ) آدرس آپی مربوط به آدرس زیر شبکه است. مثلا به صورت 223.1.0/24 نمایش میدهم یعنی ۲۴ بیت از سمت چپ مشخص کننده بخش زیر شبکه در آدرس آپی است.

در اسلاید ۴۰، به طرز عجیبی، استاد گفتند که ۷ تا زیرشبکه داریم ولی وقتی شمردند، ۶ تا بود.

سوالات

سوال ۱: فشرده سازی هدر چیست؟

سوال ۲: یک مثال از کاربرد فیلد $option$ بزنید.

آدرس دهی IP

در گذشته این آدرس دهی IP به صورت *classful* بود و بخش آدرس، مضارب بایت بود. اگر مثلاً یک بایت اول برای زیر شبکه باشد، ۲ به توان ۲۴ تا *host* می‌توانستیم داشته باشیم. یا یه بیت، یا دو بیت یا ۳ بیت برای بخش زیر شبکه می‌توانستیم داشته باشیم. و اینگونه ممکن است اتلاف داشته باشیم

$$CIDR = \text{classless interdomain routing}$$

در این روش، بخش زیر شبکه مضرب بایت نیست و می‌تواند مثلاً ۲۳ بیت باشد. که با همان 23/ نشان می‌دهیم.

سوالات مطرح شده

سوال ۱: یک *host* چگونه آدرس خود را در یک سازمان بدست می‌آورد.

دو روش آن

روش اول: به صورت *hardcore* و دستی در فایل کانفیگ وارد کنیم.

روش دوم:

استفاده از پروتکل *DHCP = dynamic host configuration protocol*: در این روش آدرس IP به طور خودکار و داینامیک از یک سرور گرفته می‌شود. هر دفعه که به سرور وصل می‌شود ممکن است یک آدرس متفاوت بگیرد و فیکس نیست. این آدرس اختصاص یافته برای مدت محدودی در اختیار اوست. *host* می‌تواند آن را تمدید کند. اگر *host* ارتباط خود را قطع کند، آدرس آزاد می‌شود و دیگران می‌توانند از آن استفاده کنند. پروتکل *DHCP* یک پروتکل *client – server* است.

مراحل مختلف پروتکل DHCP

۱. *host* درخواست *DHCP discover* خود را پرودکست می‌کند.
۲. یکی از سرورهای *DHCP* با ارسال پیام *DHCP offer* وجود خود را اعلام می‌کند و اعلام آمادگی می‌کند که می‌تواند به او آدرس IP دهد.
- این دو مرحله اول اختیاری بودند،
۳. بعد *Host* به اون *DHCP request* می‌دهد و درخواست آدرس IP می‌کند
۴. بعد سرور *DHCP* با ارسال *DHCP ACK* آدرس IP را به او می‌دهد.

سرور *DHCP* لازم نیست به صورت جداگانه طراحی شود و میتواند در روترها نیز آنها را پیکر بندی کرد و هر *interface* یک زیر شبکه را مدیریت کند.

نکات پیاده سازی مربوط به کد در پروتکل *DHCP*

در مرحله *DHCP discover* اول تمام بیت های *src* (آدرس *IP* مبدا) برابر صفر و تمام بیت های *dest* (آدرس *IP* مقصد) برابر ۱ و به معنای برودکست است. چون در ابتدا کاربر هیچ اطلاعی از شبکه ندارد. پورت مربوط به پروتکل *DHCP* برای کاربر برابر ۶۷ و برای سرور ۶۸ است. فیلد *yiaddr* در نهایت آدرسی است که به *client* داده می شود. اول صفر است. بعد یک سرور *DHCP* جواب میدهد که هست. برودکست میکند. و فیلد *yiaddr* را هم پر میکند. برای آن یک *lifetime* هم مشخص میکند. در مرحله ۳ که مرحله *DHCP request* بود، باز کاربر، فیلد *src* خود را صفر میگذارد و برودکست میکند اما اینبار *yiaddr* مساوی مقدار گرفته شده در مرحله ۲ است. بعد سرور با ارسال *DHCP ACK* با برودکست کردن، و گذاشتن *yiaddr* اعلام میکند که اوکی است. آبدی تراکنش دو پیام اول با دو پیام دوم متفاوت است.

سوالات

۱. آدرس *this – network* یا این شبکه چیست؟
 ۲. چرا مشتری در گام ۳، به جای آدرس صفر، *yiaddr* را قرار نمیدهد.
 ۳. چرا در همه ۴ پیام ارسال شده، آدرس مقصد آدرس *broadcast* میباشد؟ استدلال کنید.
- با اتصال به پروتکل *DHCP* علاوه بر آدرس *IP*، کاربر میتواند اطلاعات اضافی نیز بگیرد، مثلاً آدرس *IP* اولین روتر یا اسم و آدرس *DNS server* و پوشش شبکه. متوجه میشود که چه تعداد بیت از سمت چپ برای زیر شبکه است. پروتکل *DHCP* از *UDP* استفاده میکند. آن هم روی *IP* سوار است و آن هم روی *ETHERNET802.1*. در نهایت این فریم لایه دیتالینک است که روی شبکه میرود و برودکست میشود. آدرس برودکست با ۴۸ بیت ۱ نشان داده میشود.

روتر هم آن فریم را دریافت میکند و چون برودکست است میتواند آن را مطالعه کند. آدرس آیبی مشتری، آدرس آیبی اولین روتر در شبکه و نام و آدرس سرور *DNS* در پیام *DHCP ACK* میاید. در پروتکل استک مقصد، این فریم مرحله به مرحله دیماکس میشود.

سوالات

۲. یک شبکه چگونه آدرس زیر شبکه را بدست می‌آورد؟

هر *ISP* یک *ISP block* می‌گیرد که می‌تواند تا 2^{12} تا هاست را ساپورت کند. در واقع آدرس زیر شبکه آن 20/ می‌خورد.

اما این بلوک را می‌تواند به ۸ تا بلوک کوچک تر تبدیل کند و به دپارتمان ها بدهد. (در آنجا، 23/ می‌خورد) برای دسترسی از طریق اینترنت به آن دپارتمان ها از طریق *ISP* مربوطه شون میتوان به آنها وصل شد. اگر مثلاً یک دپارتمان از یک *ISP* به یک *ISP* دیگر برود، مدخل مربوط به *ISP* دوم تغییر نمی‌کند، و همچنین آدرس *IP* اون دپارتمان هم عوض نمیشود، فقط زیرشبکه 23/ اون دپارتمان مربوطه را در مدخل *ISP* قرار میدهیم. اگر از اینترنت کسی خواست با آن ارتباط بگیرد، چون با دو تا *ISP* اینجوری مطابق است، طبق *longest prefix match* به *ISP* دومی میرود.

ISP این بلوک را از *ICANN* می‌گیرد. اون این کار را از طریق ۵ تا *regional registry* انجام میده و هر کدام از آنها، این وظیفه را به *local registry* میدهند تا آنها تخصیص آدرس های *IP* را انجام دهند. برای حل مشکل کمبود *IP* از *NAT* استفاده میکنند. راه حل نهایی حرکت به سمت *IPv6* روی آوردند.

NAT = network address translation

با یک آدرس محلی *local network* می‌ایند یک زیر شبکه درست میکنند مثلاً با 2^8 تا هاست. بعد از طریق روتر، همه آنها را با یک آدرس *IP* مشخص به دنیا معرفی میکنند. برای تمییز هاست ها در این شبکه محلی از شماره پورت استفاده میکنند. همشون از دیدگاه اینترنت فقط یک آدرس *IP* دارند. پس لازم نیست که یک محدوده آدرس *IP* از *ISP* بگیرد. این *ip* ها میتوانند تعویض شود ولی دنیای بیرون از آن آگاه نخواهد بود. و سازمان هم میتواند به راحتی *ISP* خود را عوض کند بدون تغییر ساختار داخلی خود مربوط به آدرس دهی های شبکه محلی شون.

پس چون این *IP* های محلی از بیرون قابل مشاهده نیست، یک لایه امنیتی ایجاد کرده. در *NAT router* یک جدول نگاشت *NAT* وجود دارد. آدرس و پورت بسته ها را با آدرس خود روتر و شماره پورت جدید، جایگزین میکند. و آن را می‌فرستد. و موقع دریافت بسته های مربوط به هاست های محلی آن، باز از این جدول استفاده میکند.

پورت مربوط به *NAT router* چون ۱۶ بیت است، پس حداکثر حدود ۶۰ هزار تا تماس همزمان از شبکه خانگی میتواند رخ دهد.

مشکل های NAT:

۱. روتر باید هی تا لایه ۳ را بررسی کند.
 ۲. اینکه روتر، شماره پورت را تغییر دهد، عدول از بحث پایان پایان است چون روتر دیوایس لایه ۳ است ولی فیلد پورت برای لایه بالایی.
- مشکل اصلی کمبود آدرس با IPv6 حل می‌شود.
- بعضی از کاربرد ها در شبکه های NAT دچار مشکل میشوند و پروتکل هایی برای رفع این مشکل آمده.
- البته برای شبکه های سیار 4g و 5g کابرد خوبی دارند.

سوال

چند مورد از این کاربرد ها را نام ببرید و دو پروتکل برای آن را بگویید.

IPv6

علاوه بر اینکه ۱۲۸ بیت است بجای ۳۲ بیت در IPv4، اندازه هدر آن ثابت و ۴۰ بیت است برعکس IPv4 که متغیر بود.

علاوه بر این، از مفاهیم *flow* و *quality of service* هم پشتیبانی میکند که در IPv4 نبود این چیزا. مثلا فیلد *checksum* ندارد. تا پرودازش هدر سریعتر باشد.

فیلد آپشن حذف شده و به صورت مقداری که در *next header* قرار میگیرد، مشخص میشود. (زدم جمینای گفت مثلا به جای اینکه بده به TCP و به اون اشاره کنه و بگه بعدی هدر TCP است، میگه بعدی هدر روتینگ است) اگر هم آپشن نباشد، فیلد *next header* میگوید که به کدام پروتکل لایه بالایی تحویل دهد. (بعدی هدر TCP است) فیلد *priority* اجازه میدهد که با دیتاگرام های یک *flow* متفاوت و با تقدم متفاوت برخورد کنیم.

تعداد بیت های آدرس مبدا و مقصد هر کدام ۱۲۸ بیت هستند.

فیلد *flow label* نشان دهنده بسته های مرتب با یک *flow* یکسان را نشان میدهد. این فیلد توی همه اون ها یکسان میشود.

فیلد *hop limit* فکر کنم شبیه فیلد *TTL* است.

فیلد *payload len* هم طول پیلود را مشخص میکند.

مراجع